

Utilización de Execute

En la interfaz `Statement` hay un tercer método que podemos **utilizar para la ejecución de sentencias SQL** (`execute`), a parte de `executeQuery` (ejecución de sentencias que devuelven datos (consultas)) debemos y de `executeUpdate` (ejecución de sentencias DDL y de INSERT, DELETE, UPDATE).

Sin embargo, a veces no se conoce **el tipo de la sentencia que vamos a utilizar** y se puede utilizar:

```
Statement sent=conexion.createStatement();
boolean resultados = sent.execute(sentenciaSQL);
```

El método devuelve un valor *booleano*: *true* si la sentencia ha devuelto resultados (uno o varios objetos `ResultSet`), y *false*: si ha devuelto el número de registros afectados.

Para obtener los valores devueltos se puede utilizar : `getResultSet()` para obtener el `resultset` en el caso de que sea una consulta (como `executeQuery`) o `getUpdateCount()` para obtener el número de registros afectados (como `executeUpdate`). Ejemplo:

```
int n = sent.getUpdateCount();

0
ResultSet rs = sentencia.getResultSet();
```

A veces se obtienen varios `ResultSet` como resultado de una llamada, por ejemplo, a un procedimiento que nos devuelve varios resultados. Con `execute` nos permitiría obtener estos múltiples `ResultSet` e ir obteniéndolos utilizando el siguiente método:

```
boolean masresultados = sent.getMoreResults();
```

La llamada a este método devuelve *true* en el caso exista más `ResultSet` , y *false* en el caso contrario. Si existe, podemos consultar el nuevo `ResultSet` llamando nuevamente al método `getResultSet` y así sucesivamente.

Obtener datos de un identificador autonumérico

Cuando tenemos un valor autonumérico en un campo de una tabla, este valor no se especifica de forma manual sino que asigna de forma automática en la inserción. Si necesitamos obtener este valor (por ejemplo, para insertar este valor a continuación en otra tabla) podemos utilizar método `getGeneratedKeys` del objeto `Statement`.

```
ResultSet result = sentencia.getGeneratedKeys();
int id;
if(result.next()) {
    id = result.getInt(1);
}
```

Optimización de sentencias: PreparedStatement VS Statement

Cuando se ejecuta una sentencia SQL utilizando `Statement`, esta se compila y se manda al SGBD para su ejecución.

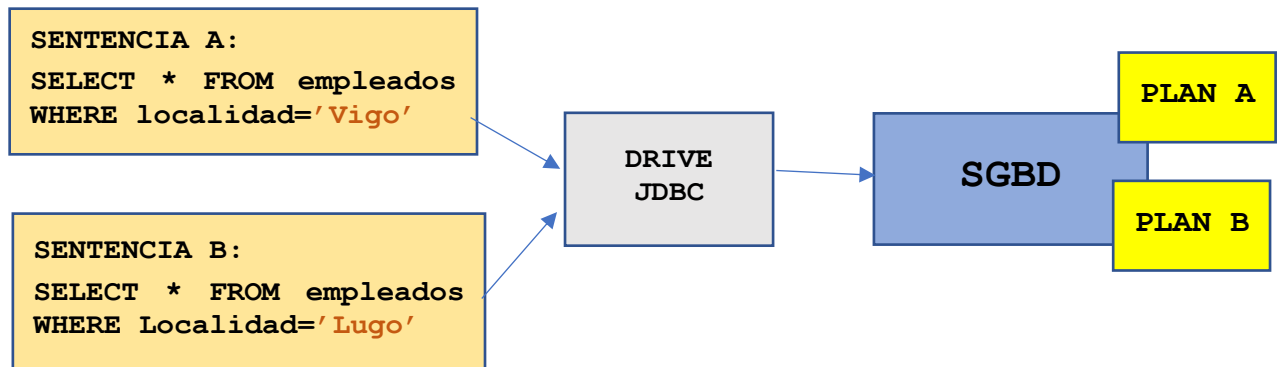
```
Statement sentencia = conexion.createStatement();
sentencia.createStatement("SELECT * FROM empleados
                           WHERE localidad='"+nombre+"'");
```

En este caso, se está construyendo una sentencia y aportando un parámetro a la consulta que se convierte en una sentencia SQL que se ejecuta vía el driver JDBC contra la base de datos.

Para cada sentencia SQL que se va a ejecutar, se construye antes un **plan de ejecución** (generado por el optimizador) en el consta la manera de ejecutar esa sentencia y a continuación se ejecuta paso a paso. El plan de ejecución es el primer paso para conocer la causa de las sentencias lentas.

Nota : A este respecto, el optimizador es muy parecido a un compilador porque traduce el código fuente (sentencia SQL) hacia un programa ejecutable (plan de ejecución).

Veamos un ejemplo de ejecución de 2 sentencias Statement para las localidades de Vigo y de Lugo. Para la ejecución de estas dos sentencias **se crean dos planes de ejecución diferentes** aunque ambas consultas sean realmente muy similares y únicamente la diferencia sea **el parámetro que se pasa**.



Para solventar la creación de planes de ejecución diferentes para sentencias SQL que se tengan que repetir varias veces y que **sean ejecutadas de forma más eficiente**, se puede dejar la sentencia **precompilada** en tiempo de creación. Para ello se utiliza la interfaz **PreparedStatement**.

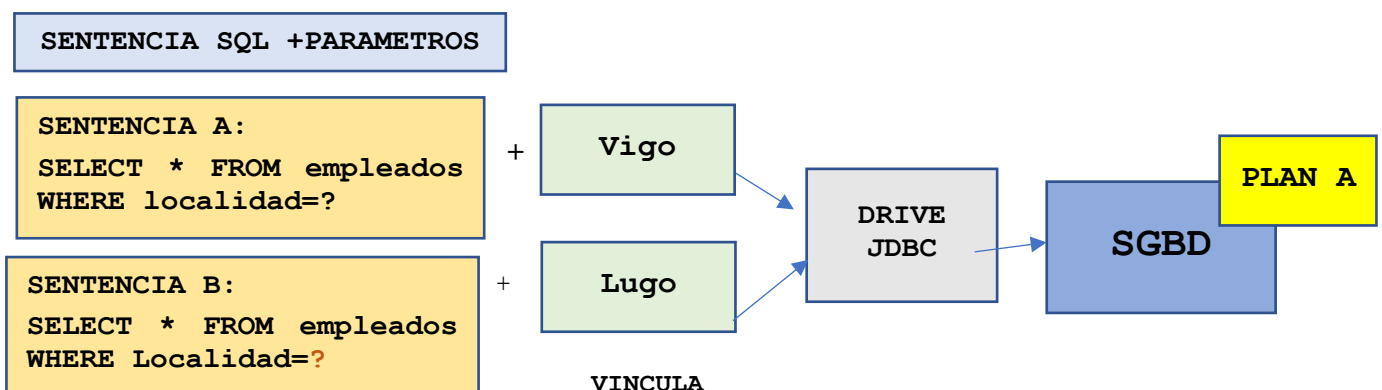
Estas estructuras permiten mantener **las consultas neutras** sin tener en cuenta los parámetros que se las pasa ya que realiza un binding de ello

```
PreparedStatement sentps = conexion.prepareStatement("SELECT * FROM empleados  
WHERE localidad=?");
```

Esta sentencia es precompilada una sola vez (sin tener en cuenta los parámetros), creando un único plan de ejecución, en lugar de ser compilada cada vez que se llama a la instrucción. Los valores se asignan en el mismo momento de ejecutar la sentencia. De este modo, cuando se trata de tareas en SQL que deben repetirse una y otra vez, las sentencias precompiladas requieren muchos **menos recursos** y son **más rápidas**.

Para dar valor a los parámetros utilizaremos los métodos **setXXX** donde **XXX** será el tipo de los datos que asignamos al parámetro, indicando el número del parámetro (que empieza desde 1) y el valor que le queremos dar. Una vez que a un parámetro se ha asignado un valor, el valor permanece hasta que lo resetee otro valor o se llame al método **clearParameters**. Ejemplo:

```
sentps.setString(1, "Vigo");  
ResultSet rs = sentps.executeQuery();
```



PreparedStatement: Evita la “injection SQL”

Un **problema de seguridad en la base de datos** que se nos puede plantear es el “**SQL injection**”. Se trata de insertar código SQL dentro de otro código SQL, para alterar su funcionamiento y conseguir que se ejecute alguna sentencia maliciosa.

Al usar **sentencias precompiladas** se fuerza que la entrada del usuario **se maneje como el contenido de un parámetro**, y no como parte del comando SQL uniendo las cadenas.

```
String s="SELECT * FROM usuarios WHERE nombre='"+nombre+"'";
```

La variable `nombre` es una cadena cuyo valor viene de un campo que es introducido por el usuario. Al introducir el usuario un nombre cualquiera, el código SQL se ejecuta. Esta opción nos permite añadir código propio que nos permita dañar o incluso permitirnos tomar el control de la BD.

Por ejemplo, si el usuario ha introducido el siguiente código: **Miguel'; drop table usuarios; select * from usuarios; grant all privileges**

Entonces el código que se ejecutaría sería la ejecución de 4 sentencias:

1. `SELECT * FROM usuarios WHERE nombre='Miguel';`
2. `drop table usuarios;`
3. `select * from usuarios;`
4. `grant all privileges`

Para evitar la inyección de SQL se recurre, en Java, a usar una sentencia preparada. De esta manera tendríamos:

```
PreparedStatement ps = con.prepareStatement(  
    "SELECT * FROM usuarios WHERE nombre=?");  
ps.setString(nombre);
```

Y la sentencia SQL que se ejecutaría, con la misma entrada que antes, es:

1. `SELECT * FROM usuarios WHERE nombre='Miguel'; drop table usuarios; select * from usuarios; grant all privileges';`